

Table of Contents

Week 6 — Summary & Examples Reference

Alternate Real-World Examples for Every Part of the Lesson

One-Line Definitions (Read These Aloud)

Part 1 Alternate Examples — What Is Inheritance?

Alternate Story — Job Roles in a Company

Alternate Story — Restaurant Menu Categories

Part 2 Alternate Examples — The IS-A Test

Alternate IS-A Test Table

Alternate IS-A vs HAS-A Story — The Taxi Example

Part 3 Alternate Examples — How Inheritance Works: Parent First

Alternate Analogy — New Phone Account Activation

Alternate Analogy — Opening a Franchise Restaurant

Part 4 Alternate Examples — What a Child Adds, What It Inherits

Alternate Inheritance Equation — Employee Hierarchy

Alternate instance of Story — ID Documents

What Is and Is Not Inherited — Applied to Employees

Part 5 Alternate Example — Java as Illustration

Concept Translation Table (Employee Version)

IS-A Test Practice Sets

Set A — Easy (different from main file)

Set B — Students Discuss

Parent-First Scenarios — Use in Class Discussion

Scenario 1 — What Breaks if the Child Goes First

Scenario 2 — What Triggers the Parent Setup

Scenario 3 — Private Salary Access

Student Discussion Questions

Common Student Confusions — Quick Fixes

Lesson Arc Summary (Mapped to Both Example Sets)

Homework Reminder

Week 6 — Summary & Examples Reference

Alternate Real-World Examples for Every Part of the Lesson

Use alongside: Week6_Inheritance.md | **Class:** Wednesday, 8 July 2026 · 6:30 PM

*This file mirrors the main lesson guide part-by-part, but every example here is **different** from the main file. Use these when a student needs a second angle, or to add variety mid-discussion without repeating yourself.*

One-Line Definitions (Read These Aloud)

Inheritance is when a child class automatically receives all shared attributes and actions from a parent class — nothing needs to be written twice.

The IS-A test asks: “Is the child truly a more specific version of the parent?” If yes, use inheritance. If no, use a field reference instead.

Parent-first means the parent’s setup always runs before the child’s own setup — like company-wide orientation before department training.

Protected (family-accessible) means children can access the parent’s data directly, but completely unrelated code cannot.

Single inheritance means a class can only have one parent class.

Part 1 Alternate Examples — What Is Inheritance?

Main file used: family traits + university people + living things + vehicle hierarchy.

This file uses: job roles in a company + a restaurant menu + passport and ID types.

Alternate Story — Job Roles in a Company

"Think about a technology company. Every single employee — whether they are a software engineer, a designer, a project manager, or an accountant — shares certain things:

- They all have a name, employee ID, department, and salary
- They all can: clockIn(), clockOut(), applyForLeave()

But each role also has something uniquely its own:

- Software Engineer adds: programmingLanguage, can: writeCode(), reviewPR()
- Designer adds: designTool, can: createMockup(), deliverAssets()
- Project Manager adds: projectCount, can: planSprint(), conductStandup()

COMPANY EMPLOYEE HIERARCHY

```
      Employee (name, employeeId, salary)
      (can: clockIn, clockOut, applyForLeave)
      /         |         \
SoftwareEngineer Designer ProjectManager
(adds: language) (adds:   (adds:
(can: writeCode) tool)    projectCount)
                        (can:   (can:
                        mockup) planSprint)
```

Without inheritance: name, employeeId, salary, clockIn(), clockOut()
written THREE times – once per role.

With inheritance: written ONCE in Employee.
All roles inherit it automatically.

Alternate Story — Restaurant Menu Categories

"A menu has categories: Food and Drink. Every Food item has: name, price, calories, isVegetarian, and can: orderItem(), addToCart(). Every Drink item also has: name, price, calories, but also: volume, isHot.

MainCourse is a type of Food. It adds: hasSide, cookingTime. Dessert is a type of Food. It adds: isGlutenFree.

The IS-A relationships here: MainCourse IS-A Food IS-A MenuItem. Dessert IS-A Food IS-A MenuItem."

MENU ITEM HIERARCHY

```
      MenuItem (name, price, calories)
      (can: order, addToCart)
      /         \
      Food       Drink
```

```

isVegetarian)    volume, isHot)
    /            \
MainCourse      Dessert
(adds:         (adds:
hasSide)       isGlutenFree)

```

Part 2 Alternate Examples — The IS-A Test

Main file used: Student/Person, Car/Engine, Library/Book table.

This file uses: transportation types, food items, and ID documents.

Alternate IS-A Test Table

IS-A TEST – TRANSPORTATION & EVERYDAY OBJECTS

Question	Answer	Decision
Is a Bus a Vehicle?	YES 	Bus inherits from Vehicle
Is a Taxi a Vehicle?	YES 	Taxi inherits from Vehicle
Is a PremiumCoffee a Coffee?	YES 	PremiumCoffee inherits from Coffee
Is a Passport an IDDocument?	YES 	Passport inherits from IDDocument
Is a NationalID an IDDocument?	YES 	NationalID inherits from IDDocument
Is a Bus a Road?	NO 	Bus TRAVELS ON a road (uses it)
Is a Taxi a Driver?	NO 	Taxi HAS-A driver inside it
Is a Coffee a Cup?	NO 	Coffee IS SERVED IN a cup
Is a Passport a Country?	NO 	Passport BELONGS TO a country

Alternate IS-A vs HAS-A Story — The Taxi Example

"A programmer builds a taxi app. They want a Taxi object to display the driver's name. They think: 'The quickest way is to make Taxi extend Driver — then the Taxi automatically has the driver's name.'

But wait: IS a Taxi a Driver? No. A taxi IS a vehicle. A driver is a separate person who works inside the taxi.

Using inheritance here would be like saying 'a classroom IS a teacher' just because a teacher works inside a classroom. Completely wrong.

The correct design: Taxi HAS-A Driver (field reference). The taxi contains a driver as one of its attributes."

WRONG: `class Taxi extends Driver` ← A taxi IS-A driver? Never.
RIGHT: `class Taxi extends Vehicle` ← A taxi IS-A vehicle 
 `class Taxi { Driver driver; }` ← A taxi HAS-A driver 

Part 3 Alternate Examples — How Inheritance Works: Parent First

Main file used: employee company-wide orientation before department training + hotel branch registering with headquarters.

This file uses: *a new mobile phone account activation + a franchise restaurant opening.*

Alternate Analogy — New Phone Account Activation

"When Sokha buys a new SIM card and activates a phone account, the first thing that happens is the network-level registration: phone number assigned, national ID verified, account created in the national database. This is common to ALL phone accounts — prepaid, postpaid, corporate.

THEN, depending on Sokha's specific plan (postpaid), her specific plan details are set up: monthly limit, billing date, roaming permissions.

The network-level setup ALWAYS comes first. Sokha's PostpaidAccount cannot be configured until the base Account exists."

PHONE ACCOUNT ACTIVATION – PARENT FIRST

Creating a PostpaidAccount for Sokha:

- Step 1: PostpaidAccount setup begins
- Step 2: "Account (parent) – here is your part:
 `phoneNumber=+855-XXX, ownerId='Sokha'"`
- Step 3: Account parent sets up: assigns number, verifies ID
- Step 4: Account parent is complete
- Step 5: PostpaidAccount continues with its own setup:
 `monthlyLimit, billingDate, roamingEnabled`
- Step 6: Complete – both parts fully set up

Alternate Analogy — Opening a Franchise Restaurant

"When a new KFC branch opens in Phnom Penh, the first thing that happens is franchise-level registration with KFC headquarters: brand name confirmed, standard menu approved, hygiene certification, uniform standards set. This is the same for EVERY KFC branch anywhere in the world.

THEN, the local branch adds its own specifics: local address, local opening hours, local manager name, local delivery zone.

Headquarters registration comes FIRST. The local details come SECOND. A branch cannot register locally before the franchise headquarters has approved and set up the common requirements."

FRANCHISE HIERARCHY – PARENT SETUP FIRST

FranchiseBranch parent: brand, standardMenu, hygieneRating

LocalBranch child: address, openingHours, managerName

Creating a new LocalBranch:

→ First: FranchiseBranch setup (brand, menu, hygiene)

→ Then: LocalBranch setup (address, hours, manager)

→ Result: a complete, valid branch ready to open

Part 4 Alternate Examples — What a Child Adds, What It Inherits

Main file used: Person → Student/Teacher equation table + instanceof explanation.

This file uses: Employee hierarchy and IDDocument hierarchy.

Alternate Inheritance Equation — Employee Hierarchy

CHILD = EVERYTHING FROM PARENT + CHILD'S OWN ADDITIONS

Employee: name, employeeId, salary
clockIn(), clockOut(), applyForLeave()
↓ (inherited automatically)

SoftwareEngineer: name, employeeId, salary + language, githubUsername
clockIn(), clockOut() + writeCode(), reviewPR()
↑
INHERITED (free) UNIQUELY THEIRS

Designer:	name, employeeId, salary	+	designTool, portfolio
	clockIn(), clockOut()	+	createMockup(), deliverAssets()
	↑		
	INHERITED (free)		UNIQUELY THEIRS

Alternate instance of Story — ID Documents

"A Passport IS-A IDDocument. So any time the system checks 'Is this an IDDocument?' — the answer for a Passport is YES. And any time it checks 'Is this a Passport?' — also YES.

But if you check 'Is this Passport a NationalID?' — No. Both are IDDocuments, but a Passport is not a NationalID."

IDDocument HIERARCHY – TWO TRUTHS AT ONCE

```
passport1 = Passport("Sokha", "KH-12345", "2033")
```

IS passport1 a Passport?	YES – it has passportNumber, expiryDate
IS passport1 an IDDocument?	YES – it also has ownerName, issuingCountry
IS passport1 a NationalID?	NO – it is a Passport, not a NationalID

Both Passport and NationalID are IDDocuments.
They are different types of the same parent concept.

What Is and Is Not Inherited — Applied to Employees

SoftwareEngineer inherits FROM Employee:

- ✓ name (protected – engineer can use directly)
- ✓ employeeId (protected – engineer can use directly)
- ✓ salary (PRIVATE – engineer CANNOT touch directly; must use Employee's getSalary() action)
- ✓ clockIn() (public – inherited for free, no rewrite)
- ✓ clockOut() (public – inherited for free, no rewrite)
- ✓ applyForLeave() (public – inherited for free, no rewrite)

SoftwareEngineer does NOT inherit:

- ✗ Employee's constructor – must trigger it with super()
- ✗ Any static members that belong to the class, not objects

Part 5 Alternate Example — Java as Illustration

Main file showed Person → Student / Teacher.

This file shows the same concept using Employee → SoftwareEngineer / Designer.

```
// PARENT - what every Employee has and can do
public class Employee {
    protected String name;           // family-accessible - children can use
    protected String employeeId;
    private double salary;           // truly private - children must use getSalary()

    public Employee(String name, String employeeId, double salary) {
        this.name = name;
        this.employeeId = employeeId;
        this.salary = salary;
    }

    public double getSalary() { return salary; } // public getter for private field

    public void clockIn() {
        System.out.println(name + " clocked in.");
    }

    public void applyForLeave() {
        System.out.println(name + " applied for leave.");
    }
}

// CHILD - SoftwareEngineer IS-A Employee
public class SoftwareEngineer extends Employee {
    private String language;

    public SoftwareEngineer(String name, String id, double salary, String language) {
        super(name, id, salary); // parent setup FIRST
        this.language = language; // then own unique setup
    }

    public void writeCode() {
        System.out.println(name + " is coding in " + language);
        // name works - it is protected in Employee
    }
}

// ANOTHER CHILD - Designer IS-A Employee
public class Designer extends Employee {
    private String designTool;
```



```

public Designer(String name, String id, double salary, String designTool) {
    super(name, id, salary);        this.designTool = designTool;    }
public void createMockup() {
    System.out.println(name + " is designing with " + designTool);    }    }

```

```

SoftwareEngineer eng = new SoftwareEngineer("Sokha", "ENG-01", 1500, "Java");
Designer          des = new Designer("Dara", "DES-01", 1200, "Figma");

eng.clockIn();           // → "Sokha clocked in."      ← INHERITED from Employee
eng.writeCode();         // → "Sokha is coding in Java" ← UNIQUE to SoftwareEngineer

des.clockIn();           // → "Dara clocked in."      ← INHERITED from Employee
des.createMockup();      // → "Dara is designing with Figma" ← UNIQUE to Designer

```

Concept Translation Table (Employee Version)

Java code	OOP concept meaning
<code>extends Employee</code>	"SoftwareEngineer IS-A Employee – inherits"
<code>super(name, id, salary)</code>	"Trigger Employee's setup first"
<code>protected name</code>	"Family-accessible – children can use directly"
<code>eng.clockIn()</code>	"Works – inherited from Employee, no rewrite"
<code>eng.writeCode()</code>	"Works – uniquely SoftwareEngineer's own action"
<code>eng instanceof Employee</code>	"True – SoftwareEngineer IS-A Employee"

IS-A Test Practice Sets

Set A — Easy (different from main file)

Pair	IS-A or HAS-A?	Why
SoftwareEngineer and Employee	IS-A	A software engineer IS a more specific type of employee
Laptop and Processor	HAS-A	A laptop contains a processor — it is not a type of processor

and <code>PhoneAccount</code>	IS-A	A postpaid account IS a type of phone account
<code>Taxi</code> and <code>Driver</code>	HAS-A	A taxi contains a driver — it is not a type of driver
<code>PremiumMembership</code> and <code>Membership</code>	IS-A	A premium membership IS a more specific type of membership
<code>School</code> and <code>Student</code>	HAS-A	A school contains students — it is not a type of student

Set B — Students Discuss

Pair	Expected Answer
<code>GraduateStudent</code> and <code>Student</code>	IS-A — a grad student IS a more specialized student
<code>Bus</code> and <code>Passenger</code>	HAS-A — a bus carries passengers, is not a type of passenger
<code>SavingsAccount</code> and <code>BankAccount</code>	IS-A — savings account IS a type of bank account
<code>Restaurant</code> and <code>Recipe</code>	HAS-A — a restaurant uses recipes, is not a type of recipe
<code>Manager</code> and <code>Employee</code>	IS-A — a manager IS a more specific type of employee
<code>Car</code> and <code>GPS</code>	HAS-A — a car may contain a GPS, is not a type of GPS

Parent-First Scenarios — Use in Class Discussion

Scenario 1 — What Breaks if the Child Goes First

Object: `SoftwareEngineer`

Setup without parent-first rule (hypothetically):

Child tries to run first:

```
language = "Java" ← fine, this is the child's own attribute
System.out.println(name + " is ready") ← BUT name doesn't exist yet!
name comes from Employee's setup – which hasn't run yet
```

Result: Error – the child tried to use something that didn't exist

This is exactly why the parent's setup must always run first.
The parent lays the foundation the child depends on.

Scenario 2 — What Triggers the Parent Setup

Object: Designer

Child constructor receives: ("Dara", "DES-01", 1200, "Figma")

The child constructor says: "super(name, id, salary)"

Translation: "Employee – here is your information. Please set
yourself up: name='Dara', id='DES-01', salary=1200"

Employee's constructor runs:

```
this.name      = "Dara"  
this.employeeId = "DES-01"  
this.salary    = 1200
```

Now Employee part is complete.

Designer continues: this.designTool = "Figma"

Full Designer object is now ready.

Scenario 3 — Private Salary Access

Object: SoftwareEngineer (extends Employee)

Attribute: salary is PRIVATE in Employee

Can SoftwareEngineer directly access salary? NO

Can SoftwareEngineer call getSalary()? YES (public getter)

Why the distinction?

salary is truly private – Employee keeps its own pay confidential,
even from its own child classes. The child must politely ask via
the public getter, just like any outside code would.

Student Discussion Questions

1. "Give a real-world example of a hierarchy you see every day — something with at least 3 levels. What does each level add that the one above it doesn't have?"

2. "In the taxi app story, the programmer wanted to make `Taxi extends Driver`. What was the real problem with that design? What should they have done instead?"
3. "Why must the parent setup run before the child's own setup? Give an example where things would break if the order were reversed."
4. "If Employee's salary is private, can a SoftwareEngineer ever know its own salary? How?"
5. "A GraduateStudent IS-A Student IS-A Person. If you ask: 'Is a GraduateStudent a Person?' — what is the answer and why? How many levels of inheritance does this cross?"

Common Student Confusions — Quick Fixes

If a student says...	Respond with...
"Inheritance copies the parent's code into the child"	"Like speaking a language your parents taught you — you didn't copy their brain, you just <i>have</i> it automatically. It's not a copy, it's an automatic possession."
"I should use inheritance whenever I want one class to have another's methods"	"Would you say a Taxi IS a Driver? No — but a Taxi HAS a Driver. For HAS-A, use a field reference, not inheritance."
"The child can access all of the parent's private data"	"Salary is private even from your own children. The child must use the parent's public <code>getSalary()</code> — not peek directly."
"Why can't a class have two parents?"	"Imagine a child class inheriting the same method name from two different parents — which version does it use? No clear answer. That's why only one parent is allowed."

Lesson Arc Summary (Mapped to Both Example Sets)

HOW THE TWO EXAMPLE SETS LINE UP, PART BY PART		
PART	MAIN FILE EXAMPLE	THIS FILE'S EXAMPLE
Part 1 – What Is Inheritance?	Family traits + Living things +	Job roles in a company + Restaurant menu

Part 2 – IS-A Test	Student/Person table + Car/Engine story	Transportation/ID table + Taxi/Driver story
Part 3 – Parent First	Company orientation + Hotel branch HQ	New phone activation + Franchise restaurant opening
Part 4 – What Child Adds/Inherits	Person→Student/Teacher equation table + instanceof on Student	Employee→Engineer/Designer equation table + instanceof on Passport
Part 5 – Java Illustration	Person → Student, Teacher classes	Employee → SoftwareEngineer, Designer

Same concepts. Different stories. Use whichever lands best.

Homework Reminder

No Java code required this week (optional bonus only).

Design a three-level university hierarchy in plain English:

1. `Person` → attributes + actions
 2. Two children: `Student` and `Teacher` — what inherited? What unique?
 3. One grandchild: `GraduateStudent` — what inherited? What unique?
 4. Apply IS-A test at each level
-

Week 6 Summary Reference | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com